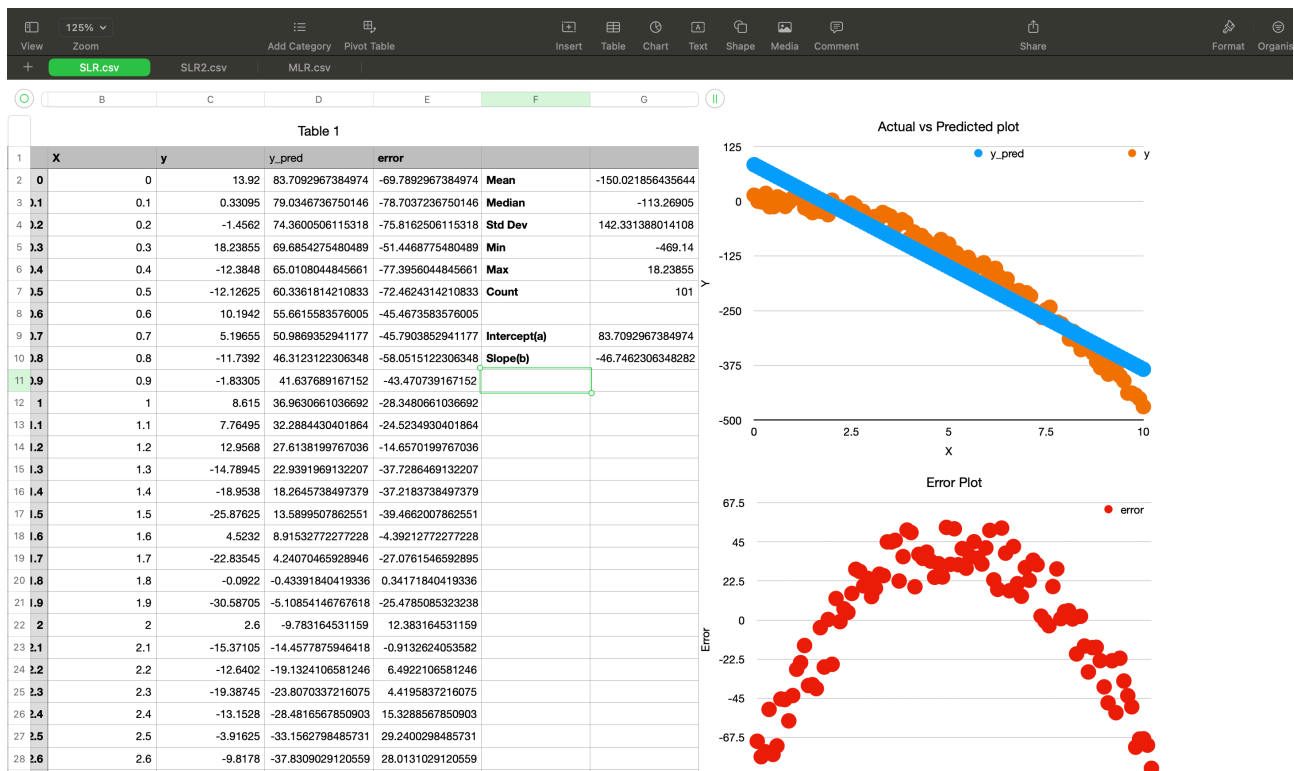
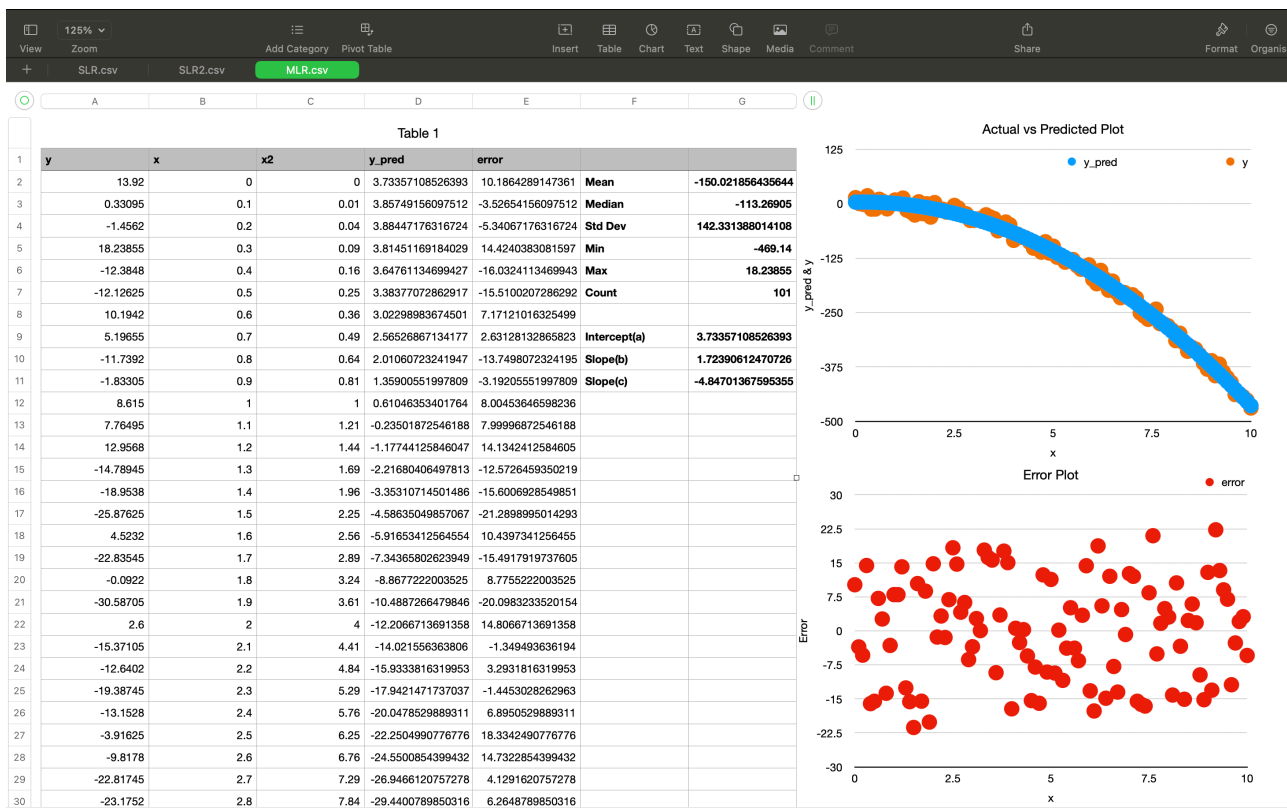


1. Carry out the following tasks related to use of Excel for Linear Regression (Note: if you use any other spreadsheet, and not Excel, these tasks are still applicable - explore that spreadsheet's in-built statistical and regression functions)
 - A. Install and enable the Data Analysis Toolpack
 - B. Load Assignment_8_newtons_law_time_distance_data_SLR.csv into the spreadsheet and visualize the data using a scatter plot, and create Descriptive Statistics for the column 'y'
 - C. Create a SLR (Simple Linear Regression) model using the data in this data file. Create the following scatter plots, analyze them, and make conclusions:
 - a) y_{actual} and $y_{\text{predicted}}$ v/s x
 - b) Error plot
 - D. Carry out the above regression steps using the following data sets, and critically analyze the outputs and errors and state your observations and conclusions:
 - c) Assignment_8_newtons_law_time_distance_data_MLR.csv
 - d) Assignment_8_newtons_law_time_distance_data_SLR2.csv

Answer:





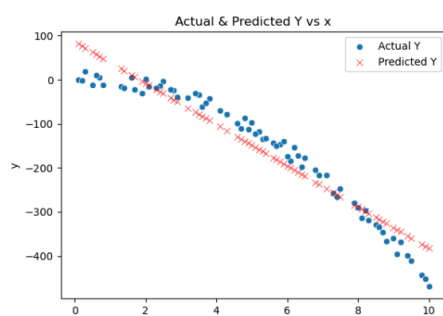
- A. Installed the data on the Numbers app in MacBook as do not have MS excel on personal machine
- B. Using the scatterplot of the Numbers app in MacBook to analyse the data. The descriptive analysis is provided as Mean, Median, Std dev, Min, Max, Count, Intercept and Slope
- C. Created a scatterplot for the actual vs predicted on the upper right hand side and the error plot on the lower right hand side
 - a. For this we first calculated the Intercept and the slope using the Number formula
 - b. Then we calculated the y_{pred} based on the value of x , intercept and the slope $y_{pred} = \text{intercept} + \text{slope} * x$
 - c. Then we calculated the error as the difference of $y - y_{pred}$
 - d. We created 2 scatterplot for the same as actual vs predicted and error plot
 - e. The Linear regression is **not a good fit** for the provided data of SLR as the variance is more and the error plot is parabolic and not randomly scattered around 0
- D. Plots for the SLR2 and MLR csv data
 - a. The SLR2 is shown plotted above. The linear regression is a **good fit** for the above as the actual fall on the predicted line and the error plot is also randomly scattered around 0
 - b. The MLR is shown plotted above. The polynomial linear regression is a **good fit** for the provided data as the actual fall on the predicted line and the error plot is also randomly scattered around 0

2. Review the Notebook Assignment_8_linear_regression_sklearn_OLS.ipynb
 - A. Focus on the plots created, understand and describe them.
 - B. Understand the different Python libraries and functions used to perform regression, and the outputs created by them
 - C. In particular review the outputs created by the OLS function, and understand the residual analysis or error analysis related statistical test results

Answer:

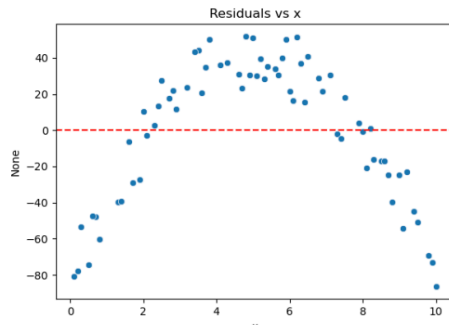
A. Understanding the plots and describing the same:

- Actual & Predicted Y vs x



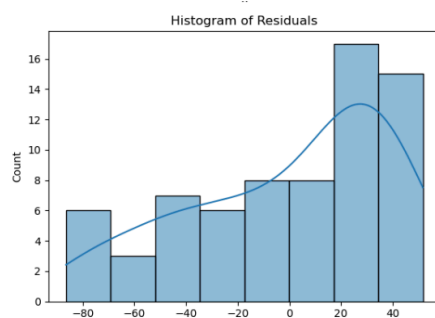
- This is being plotted using the `sns.scatterplot()` to show actual y values and predicted y values against the feature x
- The plot is showing the fitting of the linear regression model (predicted) on the specified data (actual). The following plot seems to be not completely fit the data given as the Actual data seems to be little parabolic.

• Residuals vs X



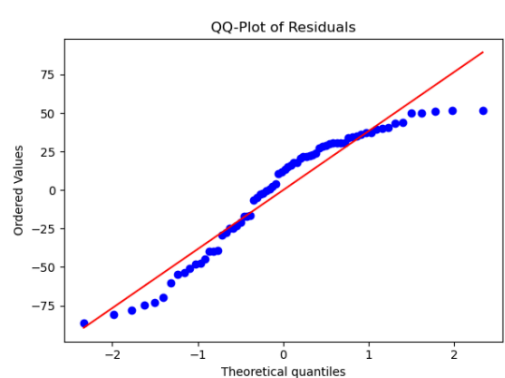
- The residual plot forms a parabola which kind of gives an intuition that the model is not a good fit.
- For a good fit model, the residual should be randomly scattered around zero.
- The applied model consistently overestimates in some regions and underestimates in others.
- For the following set of data a polynomial regression of order 2 may be a good fit

• Histogram of Residuals



- For a good model fit, the histogram of residuals should be a normal distribution curve centred at 0
- There should not be any skewness or asymmetric nature to this plot, also the tail should not be heavy and there should not be multiple peaks
- In the resulting plot we can see that the tail is heavy and the plot is skewed towards right and doesn't look like a bell curve of a normal distribution.

• QQ (Quantile-Quantile) Plot of Residuals



- For a good model fit, the points should be on the diagonal line

- In this plot the left points falls below the line, the middle points are above the line and the right tail fall below the line
- This gives an intuition of some missing variable and hence lot of outliers

B. The notebook uses 2 regressions:

- **Scikit-learn (for prediction/ML)**
 - LinearRegression
 - Train_test_split
 - mean_squared_error
 - mean_absolute_error
 - r2_score
 - model.fit()
 - model.predict()
- **Statsmodels (for statistical inference/analysis)**
 - OLSRegression
 - add_constant()
 - summary()

C. The OLS_regression function prints the standard stats models summary

```

Statsmodels OLS Regression Summary (Standard):
OLS Regression Results
=====
Dep. Variable:      y      R-squared:      0.923
Model:             OLS    Adj. R-squared:  0.922
Method:             Least Squares      F-statistic:    817.9
Date:               Sat, 18 Apr 2026    Prob (F-statistic): 1.24e-39
Time:               04:33:18          Log-Likelihood:   -355.19
No. Observations:   70              AIC:              714.4
DF Residuals:       68              BIC:              718.9
DF Model:           1
Covariance Type:    nonrobust
=====
                    coef    std err          t      P>|t|    [0.025    0.975]
-----
const      85.8900      9.641      8.909      0.000     66.651    105.129
x          -46.8585      1.638     -28.609      0.000     -50.128    -43.589
=====
Omnibus:         7.937   Durbin-Watson:      2.258
Prob(Omnibus):   0.019   Jarque-Bera (JB):    6.138
Skew:            -0.609   Prob(JB):             0.0465
---
R2: 0.9232
MAE: 33.8569
RMSE: 38.6720

```

- Coefficient table: coef, std err, t, P>|t| and confidence interval
- **Model fit statistics:**
 - **R-Squared** = 0.923
 - 92.3% of the variation in y is explained by x
 - **Ajd. R-Squared** = 0.922
 - Almost same as R²
 - Means model is not overfitting (since only 1 feature anyway)
 - **F-Statistics** = 817.9
 - **AIC** = 714.4
 - **BIC** = 718.9
 - **Log-likelihood**
- **Residual diagnostic statistics**
 - **Omnibus and prob (Omnibus)** - Do the residuals follow a normal distribution. Combines skewness (asymmetry) and kurtosis (tail heaviness)
 - Omnibus - 7.937
 - Prob(Omnibus) - 0.019
 - **Jarque-Bera and Prob (JB)** - Tests if residuals are normally distributed
 - Jarque-Bera - 6.138
 - Prob (JB) - 0.0465
 - Both test the normality of residuals
 - Since p-value < 0.05, residuals are NOT perfectly normal
 - **Skew** - measures asymmetry in residuals
 - Skew = -0.609
 - Residuals are slightly left-skewed
 - **Kurtosis** - measures tail heaviness
 - **Durbin-Watson** - checks auto-correlation in residuals
 - Durbin-Watson = 2.258
 - Ideal ~ 2
 - No autocorrelation issue

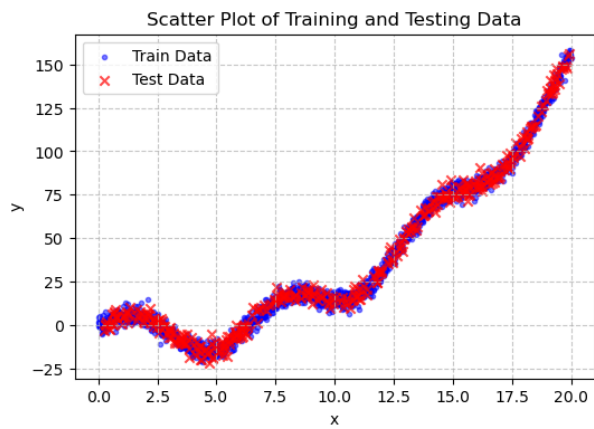
3. Review the Jupyter Notebook Assignment_8_multiple_regression_methods.ipynb, along with the data file Assignment_8_multiple_regression_methods.xlsx:

- Execute the code. Analyze the resulting plots and metrics.
- Choose the best algorithm for the given dataset. Justify your choice.

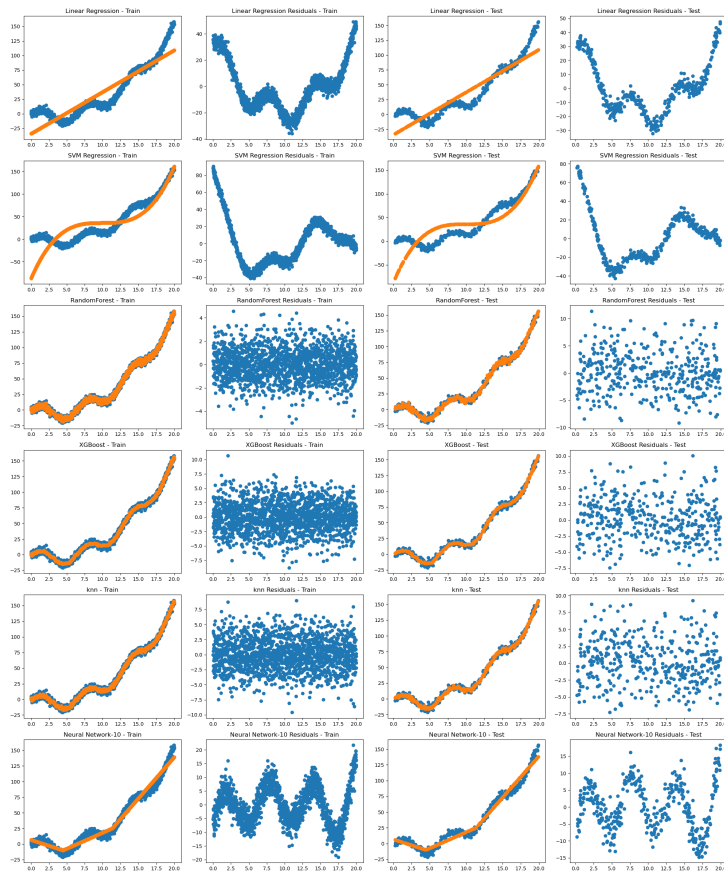
C. What is your take-away from the way the code has been structured to generate the results?

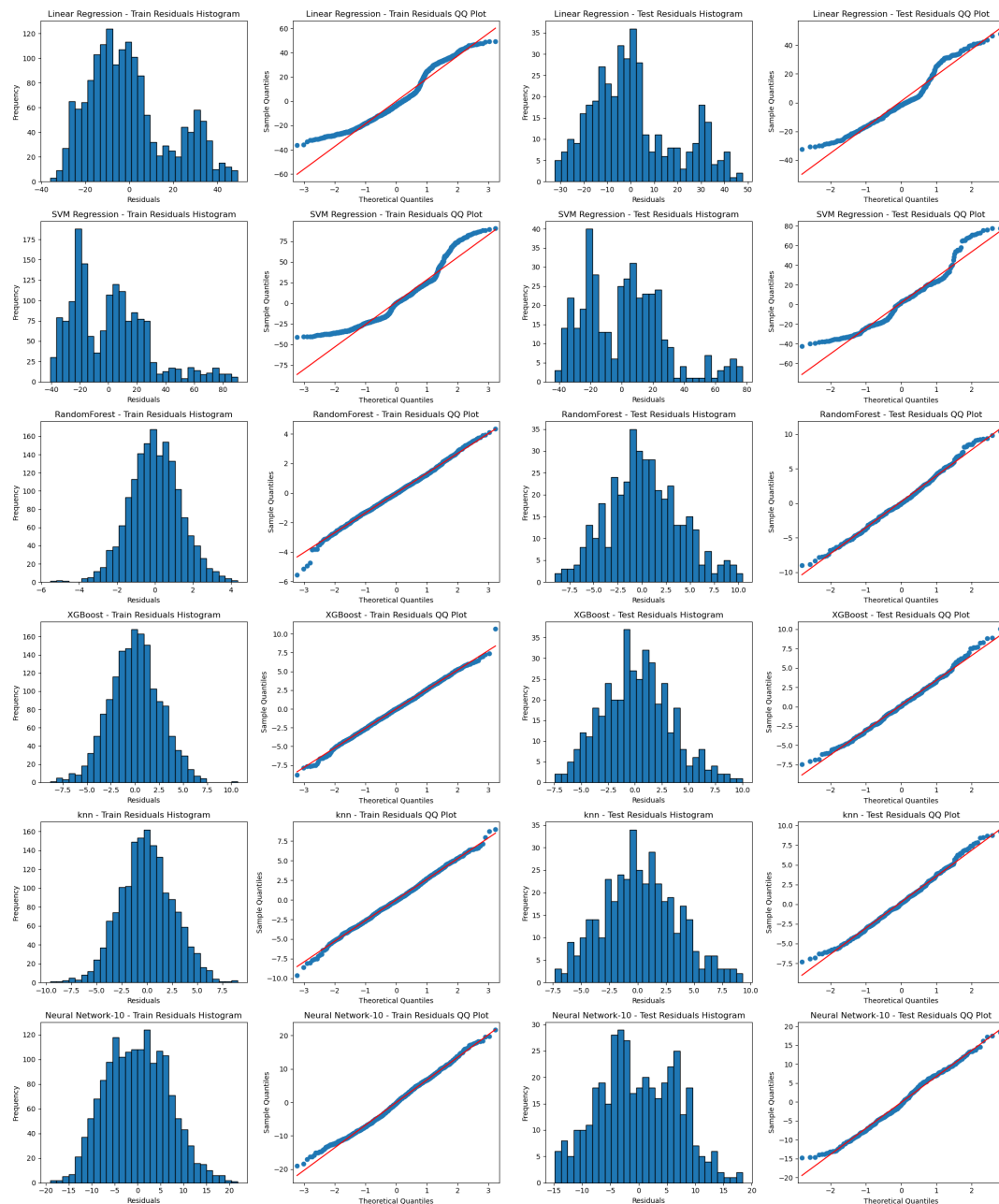
Answer:

A. Plot Analysis



- Scatter plot of training and testing data
- Grid of subplots of training and testing data by different ML algorithms followed by the residual plots





• Analysis

- The relationship between x and y is strongly non-linear
- Pattern shows initial wave oscillation
- Train and test data overlap well

• Machine learning model wise Analysis



• Linear Regression

- Predictions are straight line, cannot capture curve
- QQ plot deviates, showing non-normal residuals
- Resulting plot seems to be under-fitting
- Residuals are not randomly scattered around 0 and the histogram do not follow the gaussian distribution



• SVM Regression

- It captures some non-linearity but not fully
- The QQ plot shows deviation
- It is a Partial fit but not completely optimal
- Residuals are not randomly scattered around 0 and the histogram do not follow the gaussian distribution



• Random Forest

- Predictions closely follow the actual curve
- Residuals are randomly scattered around 0 and the histogram follows the gaussian distribution
- QQ plot doesn't show much deviation



• XGBoost (Gradient Boosting)

- The plots are very similar to the random forest
- Predictions closely follow the actual curve
- Residuals are randomly scattered around 0 and the histogram follows the gaussian distribution
- QQ plot follows the line and doesn't show any considerable deviation



- **KNN**

- The plots are similar to the XGBoost
- Predictions closely follow the actual curve
- Residuals are randomly scattered around 0 and the histogram follows the gaussian distribution
- QQ plot follows the line and doesn't show any considerable deviation



- **Neural Network**

- Captures the trend but not perfectly
- Residuals shows wave pattern, missing the expected structure
- Under-fitting or sub-optimal plots

B. Analysis of Best ML Algorithm for the given data

- **Inference** - From the initial plot analysis of the prediction plot and the residual plots the following models seems to be working well for the provided data set : **Random Forest, XGBoost & KNN**
- We must further analyse the above models with respect to the statistical OLS regression to get further statistical insights to see out of the above 3 which model is the closest statistically
 - OLS Regression Analysis (R2, RMSE, Durbin-Watson, JB p-value)

Metrics - Train Data:					
	R-squared	RMSE	Durbin-Watson	Jarque-Bera	JB P-value
Linear Regression	0.829553	18.656644	2.024608	117.827752	2.594351e-26
SVM Regression	0.638358	27.175561	1.987561	251.590113	2.332926e-55
RandomForest	0.999132	1.331492	2.053460	9.115382	1.048624e-02
XGBoost	0.996693	2.598827	2.089983	0.569595	7.521666e-01
knn	0.996598	2.635583	2.092051	0.594478	7.428664e-01
Neural Network-10	0.977496	6.779078	2.042096	16.589599	2.498126e-04
Metrics - Test Data:					
	R-squared	RMSE	Durbin-Watson	Jarque-Bera	JB P-value
Linear Regression	0.850458	18.078457	1.904427	25.441873	2.987910e-06
SVM Regression	0.689803	26.037513	1.990964	35.111435	2.374919e-08
RandomForest	0.993391	3.800531	2.086859	3.053705	2.172183e-01
XGBoost	0.995281	3.211352	2.096988	3.924603	1.405346e-01
knn	0.994983	3.311433	2.139637	4.543470	1.031331e-01
Neural Network-10	0.978289	6.888452	2.045322	6.889151	3.191832e-02

- Seeing the above the R2 and the RMSE value of XGBoost is the best R2 - 0.995 and RMSE - 3.21
- Statistical inference for Random Forest
 - the R2 is good and high, 0.993
 - JB p-value is the best among the three, 0.217
 - But the RMSE is the highest, 3.80 (slightly less accurate than others)
 - So Random forest is a well behaved model but not very precise
- Statistical inference for KNN
 - R2 = 0.994 (Very good value)
 - RMSE = 3.31 (2nd best)
 - But the R2 and RMSE are slightly low than XGBoost
 - Which makes it more sensitive to noise and data distribution
- Statistical inference for **XGBoost**
 - **Best R2 = 0.995**
 - **Lowest RMSE = 3.21**
 - **Residuals:**
 - **JB p-value = 0.140**
 - **Durbin Watson = 2 (good)**



Result - Seeing overall **XGBoost** is the best fitting model based on ML analysis and the OLS regression analysis

C. Take away on how the code is structured

- The code is very well written and follows the industry practice of writing generic code
- The analysis of the provided data is being done by the potential ML model all at once
- The plots are structured in a grid as sub plots right next to each other making it very easy to compare right next to each other
- The methods are abstracted based on the logic
- The errors are handled gracefully
- The code compares ML model plot analysis as well as the OLS regression analysis for overall seeing which model suits data the best
- Overall the code is very well written and follows the best industry practices

4. Review **sklearn** documentation for each sklearn function used in the Notebook (e.g., PolyNomialFeatures, LinearRegression, mean_squared_error, etc.,) and create a summary of each to explain the functionality, the input parameters, and the outputs. Present this in the form of a two-column Table (Function name | Description).

Answer:

Please find the functions used along with their description:

Function	Description
train_test_split	Functionality: Splits arrays or matrices into random train and test subsets for cross-validation Input parameters: • *arrays - Sequence of indexable with length/shape[0], eg: Numpy arrays, lists, scipy sparse matrices, data frames • test_size - float or int, proportion or absolute number of test samples, default = None • train_size - float or int, proportion or absolute number of train samples, default = None • random_state - int, RandomState instance or None, for reproducible shuffling, default=None • Shuffle - bool, whether to shuffle data before splitting, default=True • Stratify - array-like, for stratified splitting using class labels, default=None Output: • List containing train-test split of inputs, lenght=2*len(arrays)
LinearRegression	Functionality: Ordinary least squares Linear Regression that fits a linear model with coefficients to minimise the residual sum of squares between observed and predicted targets. Input parameters: • fit_intercept - bool, whether to calculate intercept, default=True • copy_x - bool, if True, X is copied, default=True • tol - float, precision tolerance for solution, default=1e-06 • n_jobs - int, number of jobs for computation, default=None • positive - bool, forces positive coefficients, default=False Output: • Fitted estimator with attributes like coef_ (estimated coefficients), intercept_ (independent term), rank_ (rank of matrix x), singular_ (singular values), etc., • Methods include fit(X, y, sample_weight=None) to fit the model, predict(X) to predict using the model, score(X, y, sample_weight=None) to return R² score
mean_absolute_error	Functionality: Computes the mean absolute error regression loss, measuring the average of the absolute errors between predicted and true values. Input parameters: • y_true - array-like of shape (n_samples) or (n_samples, n_outputs), estimated target values • y_pred - array-like of shape (n_samples) or (n_samples, n_outputs), estimated target values • sample_weight - array-like of shape (n_samples), default=None, sample weights • multioutput - {'raw_values', 'uniform_average'} or array-like of shape (n_outputs), default='uniform_average'. Defines aggregation of multiple output values Output: Float or array of floats (loss value(s), non-negative, best value is 0.0)

r2_score	Functionality: Computes the R^2 (coefficient of determination) regression score, measuring the proportion of variance in the dependent variable that's predictable from the independent variables. Best score is 1.0, can be negative. Input parameters: • y_true - array-like of shape (n_samples) or (n_samples, n_outputs), estimated target values • y_pred - array-like of shape (n_samples) or (n_samples, n_outputs), estimated target values • sample_weight - array-like of shape (n_samples), default=None, sample weights • multioutput - {'raw_values', 'uniform_average'} or array-like of shape (n_outputs), default='uniform_average'. Defines aggregation of multiple output values • force_finite - bool, default=True, replaces NaN with real numbers or constant data. Output: Float or ndarray of floats (the R^2 score)
PolynomialFeatures	Functionality: Generates polynomial and interaction features from input features to enable non-linear modelling. Input parameters: • degree – int, degree of polynomial features • interaction_only – bool, only interaction terms if True • include_bias – bool, include bias column (1s), default=True • order – str, memory order ('C' or 'F') Output: Transformed feature matrix with polynomial combinations Methods: fit(X), transform(X), fit_transform(X)
StandardScaler	Functionality: Standardises features by removing mean and scaling to unit variance. Input parameters: • copy – bool, copy input data, default=True • with_mean – bool, center data before scaling, default=True • with_std – bool, scale to unit variance, default=True Output: Scaled data with mean ≈ 0 and variance ≈ 1 Attributes: mean_, scale_ Methods: fit(X), transform(X), fit_transform(X)

SVR	Functionality: Support Vector Regression using kernel-based methods to model non-linear relationships. Input parameters: • kernel – str, type of kernel ('linear', 'rbf', etc.), default='rbf' • degree – int, degree for polynomial kernel • C – float, regularization parameter • epsilon – float, margin of tolerance • gamma – scale of kernel influence Output: Fitted SVR model Methods: fit(X, y), predict(X)
KNeighborsRegressor	Functionality: Predicts values based on the average of nearest neighbors. Input parameters: • n_neighbors – int, number of neighbors • weights – 'uniform' or 'distance' • algorithm – search algorithm ('auto', 'ball_tree', etc.) • p – int, power parameter for distance metric Output: Fitted KNN model Methods: fit(X, y), predict(X)
RandomForestRegressor	Functionality: Ensemble method that builds multiple decision trees and averages their predictions. Input parameters: • n_estimators – int, number of trees • criterion – function to measure split quality • max_depth – int, max tree depth • min_samples_split – int, minimum samples to split • random_state – int, seed for reproducibility Output: Fitted ensemble model Attributes: feature_importances_ Methods: fit(X, y), predict(X)
GradientBoostingRegressor	Functionality: Builds models sequentially where each new model corrects errors of previous ones. Input parameters: • loss – loss function ('squared_error', etc.) • learning_rate – shrinkage factor • n_estimators – number of boosting stages • max_depth – depth of individual trees • random_state – seed Output: Boosted regression model Methods: fit(X, y), predict(X)

MLPRegressor	Functionality: Feedforward neural network for regression tasks. Input parameters: • hidden_layer_sizes – tuple defining layers • activation – activation function ('relu', 'tanh', etc.) • solver – optimization algorithm ('adam', 'sgd') • max_iter – max training iterations • learning_rate – learning rate schedule Output: Trained neural network model Methods: fit(X, y), predict(X)
--------------	--